TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 6: Inside the xv6 File System

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Reading the filesystem listing **5**

1.1 File types and root identity 5

1.2 An empty directory 5

2 Implementing mystat and observing hard links **6**

2.1 Interface and error handling 6

2.2 Metadata before and after linking 6

2.3 Unlinking and freeing blocks 7

2.4 What an on-disk inode stores 7

3 Measuring filesystem limits **9**

3.1 Fixed-size names 9

3.2 Maximum file length 9

3.3 Disk layout from mkfs 9

4 Implementing and testing find **11**

4.1 Traversal and resource handling 11

4.2 Required test on a fresh image 11

4.3 Optional inode numbers 12

4.4 Additional checks 12

Conclusions **13**

ABSTRACT

This laboratory examines the on-disk structures and visible behaviour of the xv6 filesystem. Two user programs expose inode metadata and recursively search directory trees. Fresh-image experiments demonstrate hard links, fixed-length names, directory entry sizes and inode identity. Source inspection explains block reclamation and storage limits, while traversal tests exercise descriptor management and the optional inode display.

Keywords: xv6, inode, directory, hard link, recursive traversal, filesystem

INTRODUCTION

This laboratory examines directory entries, inode metadata, hard links and storage limits in xv6. Two user programs were implemented: `mystat`, which prints inode information, and `find`, which recursively searches by the last pathname component. The optional `-n` form of `find` also prints inode numbers.

The work uses the same course revision as Lab 4, `06aad25c735fd3159bdfae5680be4eab7b1668b2`, with the RISC-V GCC toolchain and QEMU on openSUSE Tumbleweed. A separate working copy was built from `~/xv6-riscv`. Both experiments began with freshly built filesystem images: the first covers inspection and link removal, and the second contains the exact directory tree prescribed for `find`. QEMU was exited with Ctrl-A followed by X between runs.

The source files are submitted as `user/mystat.c` and `user/find.c`, together with the full Makefile and a focused Makefile patch. The command transcripts and relevant kernel-source excerpts are retained in `evidence/`.

1 READING THE FILESYSTEM LISTING

1.1 File types and root identity

The first `ls` produced the following selected entries:

```
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 7 35368
console         3 27 0
```

Figure 1.1 – Directory listing with types, inode numbers and sizes

The columns are name, type, inode number and size in bytes. `kernel/stat.h` defines **1 as a directory** (`T_DIR`), **2 as a regular file** (`T_FILE`) and **3 as a device** (`T_DEVICE`). The console is therefore a device inode rather than a regular file holding console contents.

The root directory has inode **1**, matching `ROOTINO` in `kernel/fs.h`. At root, `.` denotes the current directory and `..` denotes its parent, which is root itself. Both therefore identify inode 1. The executable and console inode numbers depend on the files included in the image; the handout's sample numbers are not fixed requirements.

1.2 An empty directory

```
$ mkdir a
$ ls a
.               1 28 32
..              1 1 1024
$ mystat a
a: directory, inode 28, links 1, size 32 bytes
```

Figure 1.2 – Entries and size of an empty directory

A directory entry consists of a two-byte `ushort inum` and a 14-byte name array, so `sizeof(struct dirent)` is **16 bytes**. Directory `a` contains two entries, `.` and `..`, accounting for `2 * 16 = 32` bytes. Its `.` entry identifies inode 28, and `..` identifies the parent, inode 1. The 1024 displayed beside `..` is the parent directory's size, not the length of that single entry.

2 IMPLEMENTING MYSTAT AND OBSERVING HARD LINKS

2.1 Interface and error handling

`mystat` loops over every pathname operand and calls the user-library `stat` function. Its type helper maps the three defined types to `directory`, `file` and `device`, returning `?` for an unknown value. Successful calls print the requested name, type, inode number, link count and size. Size is printed with `%lu`, supported by this revision's `user/printf.c`, preserving the `uint64` value in `struct stat`.

A failed lookup prints a diagnostic to descriptor 2 and processing continues with later operands. The program exits with status 1 if any lookup failed, and status 0 otherwise. Calling it without operands prints a usage message and exits with status 1. This extends the handout's basic loop with useful argument and failure reporting.

The Makefile retains the earlier user utilities and adds tab-indented continuation lines for `$U/_mystat` and `$U/_find` in `UPROGS`. A clean build of `kernel/kernel` and `fs.img` completed successfully.

2.2 Metadata before and after linking

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Figure 2.1 – Hard-link counts before and after unlinking

Creating `r2` adds a second directory entry pointing to inode 2 and increases its link count from 1 to 2. Removing `README` decreases the count to 1. The remaining name still identifies inode 2, the file remains 2441 bytes long, and its contents remain readable: `wc` counted 48 lines, 336 words and 2441 bytes. A hard link does not copy the data.

Root's link count is 2 in this run because `mkdir a` already added a child directory. This differs from the handout's initial example with link count 1. xv6 deliberately does not count a directory's own `.` entry as a separate link; directory creation increases the parent's count for the child's `..` entry. Therefore an empty

xv6 directory has link count 1 even though it contains two directory entries.

Error handling was also observed:

```
$ mystat missing r2
mystat: cannot stat missing
r2: file, inode 2, links 1, size 2441 bytes
$ mystat
usage: mystat <path> [path ...]
```

Figure 2.2 – Metadata lookup diagnostics

2.3 Unlinking and freeing blocks

In the pinned `kernel/sysfile.c`, the file's link count is decremented by **`ip->nlink--;` at line 244** in `sys_unlink`. The following `iupdate(ip)` persists the inode metadata. The earlier `dp->nlink--` at line 239 instead updates a parent when a directory is removed; these two counters must not be confused.

The file's data blocks are freed only after its link count is zero and its final in-memory reference is being released. In `kernel/fs.c`, `iput` tests `ip->ref == 1 && ip->valid && ip->nlink == 0` at line 357, then calls `itrunc(ip)` at line 365. `itrunc` frees direct data blocks, indirect data blocks and the indirect-address block. This revision then calls `ifree` to make the inode reusable. An open descriptor can delay this reclamation after the last directory name has been removed. After only `rm README`, the surviving `r2` link already prevents reclamation.

2.4 What an on-disk inode stores

`struct dinode` in `kernel/fs.h` contains:

Field	Meaning
`short type`	Directory, regular file, device, or unused inode
`short major`	Device major number, for a device inode
`short minor`	Device minor number, for a device inode
`short nlink`	Number of counted directory links
`uint size`	Logical size in bytes
`uint addrs[NDIRECT + 1]`	Twelve direct addresses and one indirect-block address

The structure occupies 64 bytes: four two-byte fields, one four-byte size and thirteen four-byte addresses. A filename lives in a directory entry, not in this inode. This xv6 revision has no inode ownership,

Unix permission-mode fields, timestamps or symbolic-link type. The `dev` and `ino` reported by `struct stat` identify an inode's location; they are not fields stored inside `struct dinode`.

3 MEASURING FILESYSTEM LIMITS

3.1 Fixed-size names

The constants in `kernel/fs.h` are `BSIZE = 1024`, `NDIRECT = 12`, `NINDIRECT = BSIZE / sizeof(uint)` and `DIRSIZ = 14`.

```
$ mkdir averyveryverylongname
$ ls
averyveryveryl 1 29 32
$ mystat averyveryverylongname averyveryveryl
averyveryverylongname: directory, inode 29, links 1, size 32 bytes
averyveryveryl: directory, inode 29, links 1, size 32 bytes
```

Figure 3.1 – Directory-name length and lookup

The `ls` excerpt selects the newly created directory. Its stored name is **`averyveryveryl`**, the first 14 characters. `DIRSIZ` explains the truncation. Path lookup likewise uses the first 14 characters of a component, so both spellings identify inode 29. A full-length 14-byte name need not contain a terminating NUL inside the directory entry, which matters when passing it to string functions.

3.2 Maximum file length

One indirect block holds `1024 / 4 = 256` block addresses. Together with twelve direct blocks, the inode addresses 268 data blocks:

```
NINDIRECT = 1024 / 4 = 256
MAXFILE   = 12 + 256 = 268 data blocks
maximum   = 268 * 1024 = 274432 bytes = 268 KiB
```

Figure 3.2 – Maximum file-size calculation

This is the structural maximum file-data length. The indirect-address block consumes another disk block but does not add to the file's logical contents. A nearly full disk may run out of free blocks before a file reaches this limit. The calculation follows the source constants; it is not a claim that a maximum-size file was written during this exercise.

3.3 Disk layout from mkfs

The clean build printed:

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1102 blocks have been allocated
```

Figure 3.3 – Filesystem allocation reported by mkfs

The disk contains **2000 blocks**, each 1024 bytes, for **2,048,000 bytes** overall. Its contiguous layout is:

Table 3.1 – Filesystem image layout

Region	Block numbers	Block count
Boot	0	1
Superblock	1	1
Log	2–32	31
Inodes	33–45	13
Free-block bitmap	46	1
Data area	47–1999	1953

The metadata total is `1 + 1 + 31 + 13 + 1 = 47`; `2000 - 47 = 1953` blocks remain in the data region. The log allocation includes its header. The 1953 data-area blocks are not all initially free: the root directory, README, executables and any indirect-address blocks already occupy part of them. Immediately after this build, the first 1102 disk blocks were marked allocated, leaving 898 blocks unallocated before the boot-time and user-created changes.

4 IMPLEMENTING AND TESTING FIND

4.1 Traversal and resource handling

`find <dir> <name>` compares the final component of each visited pathname with the requested name using `strcmp`. A matching directory is printed as well as searched, and a matching regular file is printed without being traversed. A starting file operand is also accepted. Trailing slashes on the starting pathname are normalized except at root.

For a directory, the program reads fixed-size `struct dirent` records, skips entries with `inum == 0`, and excludes `.` and `..` from recursion. Each name is copied into a `DIRSIZ + 1` buffer and explicitly NUL-terminated. Child paths include a separator only when required, and paths exceeding the 512-byte buffer policy, including the terminator, produce a diagnostic.

Before recursion, the names are retained in a heap-allocated linked list and the directory descriptor is closed. Child pathnames are also allocated on the heap and freed after use. This avoids retaining one open descriptor per ancestor: xv6's `NOFILE` limit is 16, with standard descriptors already occupying three slots. Directory read errors, failed lookup/open calls and allocation failures are reported to standard error and result in a nonzero final status.

The traversal targets the stable directory trees in this experiment. Its name snapshot is not an atomic filesystem snapshot; concurrent renames can cause lookup errors. Recursion still consumes the finite xv6 user stack, so the program does not claim unlimited depth.

4.2 Required test on a fresh image

After exiting QEMU, `make clean` and a new build created a fresh image, removing the earlier README-link experiment. The required commands then produced:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

Figure 4.1 – Recursive filename search

All three matching files were found, and searching for `b` found the matching directory. Excluding `.`

and `..` prevents cycling back to ancestors. The two nested files each contain a letter and newline, while the root `r2` is a hard link to the executable `cat`.

4.3 Optional inode numbers

```
$ find . r2 -n
./a/b/r2 30
./a/r2 31
./r2 7
$ mystat cat r2 a/r2 a/b/r2
cat: file, inode 7, links 2, size 35368 bytes
r2: file, inode 7, links 2, size 35368 bytes
a/r2: file, inode 31, links 1, size 2 bytes
a/b/r2: file, inode 30, links 1, size 2 bytes
```

Figure 4.2 – Matching paths and inode identities

Only `./r2` is the same file as `cat`: both identify inode 7 and report two links. The other two paths have distinct inodes, 30 and 31, even though all three last components are named `r2`.

4.4 Additional checks

A missing path produced `find: cannot stat missing`; invalid argument counts and an unknown fourth argument produced the usage message. A nonexistent target name produced no match lines. The following cases also succeeded: `find a/ r2`, `find a/r2 r2`, `find /a r2`, a directory with a full 14-character name, and a directory containing a slot freed by removing a file.

A deeper tree contained 19 nested directories, exceeding the per-process descriptor count. Both ordinary and inode-number searches reached its `needle` file without an open failure. A second tree contained 24 sibling directories; repeated searches found `wide/d23/needle`. These results support the descriptor-closing strategy and the handling of both deep and wide trees. Full commands and output are retained in `evidence/5-edge-cases.txt` and `evidence/6-depth-width.txt`.

CONCLUSIONS

The listing and `mystat` results connect names to inode identities and link counts. Removing one hard link leaves the inode and its contents reachable through another, while source inspection explains when the final unlink can reclaim storage. The filesystem constants account for 14-byte names, 16-byte directory entries and a 268 KiB maximum file size. Finally, `find` turns the directory-entry representation into a working recursive traversal, and its inode display distinguishes equal names from references to the same file.